

UNIVERSIDAD DON BOSCO

Diseño y Programación de Software Multiplataforma

Investigación aplicada: React Native y Figma en NutriCare

Avance 1 - versión móvil funcional

Estudiante	**Fátima Yesenia Oviedo Martínez - OM160481**
Docente	**Alexander Alberto Siguenza**
Proyecto	**NutriCare - seguimiento nutricional multiplataforma**
Fecha	**18 de octubre de 2026**

Documento técnico con investigación, decisiones UX/UI, evidencia de implementación y trazabilidad con la rúbrica del nivel Estratégico (9.00-10.00).

1. Resumen ejecutivo y alcance del avance

NutriCare responde a una necesidad real: centralizar el acompañamiento entre nutricionista y paciente. Esta entrega convierte los mockups móviles de la Fase 1 en una aplicación React Native navegable, con autenticación local demostrable, formularios validados, vistas diferenciadas por rol y selección de fotografías. La solución conserva el lenguaje visual del proyecto web y de Figma: fondos claros, verde turquesa como color de acción, acento lima, tarjetas redondeadas y jerarquía tipográfica marcada.

Resultado verificable
El proyecto móvil compila con TypeScript en modo estricto y Expo generó correctamente el paquete web: 934 módulos, 17 bundles de rutas y las pantallas de acceso, registro, recuperación, inicio, plan, progreso, perfil y fotografía.

Trazabilidad inmediata

Requisito del avance	Implementación en NutriCare	Estado
Login y registro funcional	AuthContext, AsyncStorage, pantallas login/register y sesión persistente	Cumplido
Módulo principal navegable	Expo Router con Stack, grupos de rutas y navegación inferior por pestañas	Cumplido
Coherencia con Figma	Tokens de color/espacio/radio, tarjetas, jerarquía y flujos de Fase 1	Cumplido
Validaciones de formularios	React Hook Form + Zod: correo, contraseña, coincidencia y términos	Cumplido
Investigación APA 7	Marco teórico, aplicación práctica, decisiones y referencias oficiales	Cumplido

Criterio de éxito

- La aplicación no se limita a pantallas estáticas: mantiene sesión, cambia la experiencia según el rol y conecta acciones con rutas reales.
- La arquitectura separa interfaz, estado, validación y datos de demostración, facilitando la futura integración con Firebase.
- Las decisiones se justifican con documentación oficial de React Native, Expo, Figma y W3C.

2. Del problema al producto móvil

La Fase 1 definió a NutriCare como una solución multiplataforma para el negocio Nutrixión Consciente. El problema central es la dispersión de planes, mediciones, citas y evidencia fotográfica. El móvil prioriza la consulta cotidiana del paciente, sin eliminar la visibilidad rápida que necesita la nutricionista (Oviedo Martínez, 2026).

Usuarios y tareas críticas

Rol	Necesidad	Respuesta en este avance
Paciente	Consultar su plan y entender su avance	Inicio personalizado, plan diario, gráfica de peso, IMC y meta
Paciente	Registrar evidencia de comidas	Selección de imagen, tipo de comida, comentario y límite de 10 MB
Nutricionista	Observar estado general	Resumen de agenda, fotografías pendientes, pacientes y evolución
Ambos	Acceder de forma comprensible	Login, registro, recuperación y perfil adaptados al rol

Proceso Figma -> código

- 1. Comprender: se retomó la necesidad, roles y arquitectura UI/lógica/datos de la Fase 1.
- 2. Comparar: cada mockup se descompuso en componentes repetibles (encabezado, tarjeta, campo, botón y navegación).
- 3. Sistematizar: colores, espacios y radios se convirtieron en tokens dentro de constants/theme.ts.
- 4. Implementar: las pantallas se construyeron con componentes nativos y rutas basadas en archivos.
- 5. Verificar: se ejecutó TypeScript y una exportación completa de Expo para detectar rutas o tipos rotos.

Decisión de alcance
Para el avance se usa persistencia local en AsyncStorage con cuentas de demostración. Esto hace comprobable el flujo sin depender de red. Firebase Authentication y Firestore quedan documentados como evolución para la entrega integrada; no se presenta la persistencia local como seguridad de producción.

3. React Native aplicado a NutriCare

React Native es un framework abierto para crear aplicaciones Android e iOS mediante React y capacidades nativas. La interfaz se expresa con componentes reutilizables y anidables; View, Text, Image, TextInput, Pressable y ScrollView reemplazan los elementos HTML habituales (Meta Platforms, s. f.-a; Meta Platforms, s. f.-b). En NutriCare esto permite compartir principios y lógica con la versión web sin copiar su marcado.

Conceptos fundamentales

Concepto	Qué significa	Ejemplo en el proyecto
Componente	Bloque reutilizable con interfaz y comportamiento	BrandHeader, FormField, PrimaryButton, SurfaceCard
Props	Datos que el componente padre entrega al hijo	label, error, role, loading y variant
Estado	Datos que cambian durante la interacción	sesión, imagen elegida, comentario y comida activa
Context	Estado compartido sin pasarlo pantalla por pantalla	AuthProvider expone user, login, register y logout
StyleSheet/Flexbox	Estilos y distribución multiplataforma	filas de estadísticas, tarjetas flexibles y navegación

Elección tecnológica

- Expo SDK 58 reduce configuración nativa inicial y mantiene un proyecto universal Android, iOS y web.
- React Native 0.88 y React 19.3 proveen la base de componentes y estado.
- TypeScript agrega contratos estáticos; Expo recomienda comprobación con tsc y admite alias de rutas (Expo, s. f.-c).
- El modo strict está habilitado para encontrar errores antes de ejecutar la aplicación.
- Los componentes UI internos evitan duplicación y hacen que un cambio visual se propague de forma consistente.

```
// Un componente reusable recibe propiedades tipadas
<PrimaryButton
label="Guardar fotografía"
loading={isSubmitting}
onPress={save}
/>
```

4. Figma como puente entre diseño e implementación

Figma no se utilizó únicamente para dibujar pantallas. Funcionó como especificación visual: los estilos reutilizables permiten mantener propiedades coherentes y los componentes actúan como bloques de un sistema de diseño (Figma, s. f.-a; Figma, s. f.-b). Esta lógica se trasladó al código mediante tokens y componentes.

Decisión de Figma	Token o componente en React Native	Impacto
Verde de acción	colors.primary = #07866F	Botones, pestaña activa y énfasis
Verde lima	colors.lime = #D9F39A	Metas y llamadas de atención positivas
Superficies claras	background y surface	Lectura limpia y baja carga visual
Esquinas redondeadas	radius sm/md/lg/pill	Consistencia entre campos, tarjetas y etiquetas
Escala de separación	spacing xs a xxl	Ritmo estable sin valores arbitrarios repetidos
Elementos recurrentes	FormField, SurfaceCard, BrandHeader	Mantenimiento y reutilización

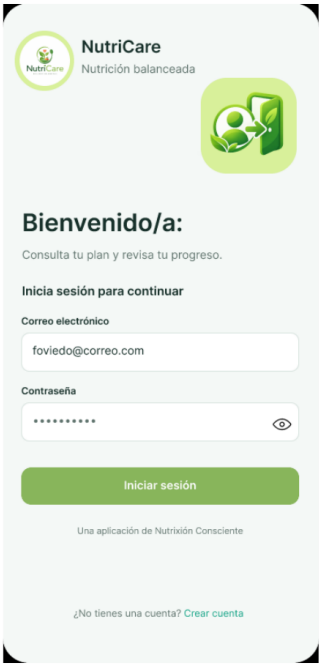
Criterios de traducción

- Se preserva la jerarquía, no cada píxel: título grande, dato principal, ayuda secundaria y acción dominante.
- Se usan componentes nativos para que el comportamiento táctil, teclado y scroll se sientan propios del dispositivo.
- Los textos largos pueden crecer; las tarjetas y pantallas utilizan flujo vertical en vez de alturas rígidas.
- El diseño base se conserva en ambos roles, pero el contenido se adapta al objetivo del usuario.

Justificación
La coherencia visual se obtiene al compartir decisiones, no al copiar capturas. Los tokens hacen comprobable la relación entre Figma y código, y reducen divergencias futuras.

5. Evidencia visual de la Fase 1 utilizada

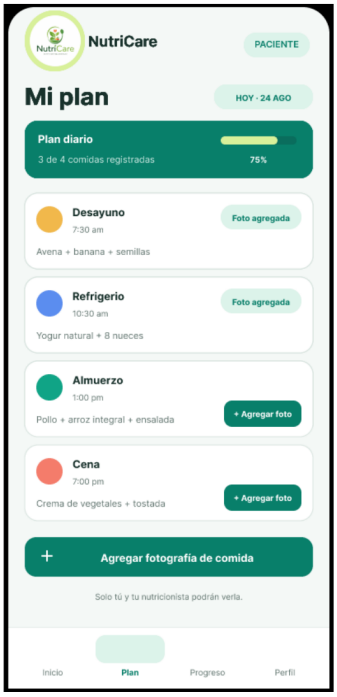
Los siguientes mockups pertenecen al documento de diseño de la Fase 1. Sirvieron como referencia para autenticación, inicio y plan. La implementación conserva identidad, jerarquía, tarjetas y navegación inferior, a la vez que incorpora estados reales, validación y rutas.



Acceso



Inicio del paciente



Plan diario

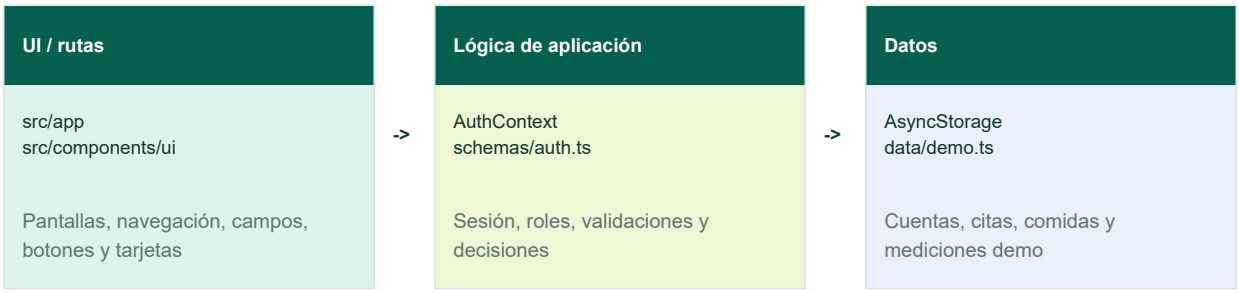
Correspondencia implementada

Mockup	Ruta	Mejora funcional incorporada
Inicio de sesión	/login	Errores por campo, contraseña oculta, estado de envío y accesos demo
Home paciente	/	Contenido por rol, meta semanal y acceso al plan completo
Plan	/plan	Lista de comidas, estado de foto y navegación a carga
Progreso	/progress	Gráfica, cambio total, meta e IMC
Subir fotografía	/upload-photo	Permiso, selección, tamaño máximo, comentario y confirmación

Fuente de las imágenes: Oviedo Martínez (2026), Proyecto Fase 1 NutriCare.

6. Arquitectura de la aplicación móvil

La organización sigue la arquitectura de tres capas definida en la Fase 1, adaptada al alcance actual. Las rutas renderizan la interfaz, el contexto coordina la sesión y los esquemas validan entradas; la persistencia local actúa como repositorio temporal.



Estructura principal

- mobile/src/app - rutas y pantallas; los grupos (auth) y (tabs) organizan sin alterar la URL.
- mobile/src/components/ui - piezas visuales reutilizables.
- mobile/src/context - estado global de autenticación.
- mobile/src/schemas - reglas Zod para login, registro y recuperación.
- mobile/src/constants - tokens del sistema visual.
- mobile/src/data - datos controlados para la demostración.
- mobile/src/types - contratos TypeScript de cuenta y sesión.

Por qué Expo Router

Expo Router convierte archivos dentro de src/app en rutas, ofrece navegación universal y simplifica el modelo mental del proyecto. Las rutas agrupadas y los archivos _layout separan los flujos de autenticación y las pestañas principales. Expo recomienda este enfoque para proyectos nuevos y permite rutas tipadas (Expo, s. f.-a; Expo, s. f.-b).

7. Autenticación, sesión y validación

El flujo de acceso está diseñado para demostrar comportamiento completo sin servidor. AuthProvider encapsula las operaciones de registro, inicio y cierre; AsyncStorage conserva la sesión entre reinicios. Expo describe AsyncStorage como almacenamiento clave-valor persistente y no cifrado, apropiado para estado pequeño no sensible (Expo, s. f.-d).

Flujo de inicio de sesión

1. Captura	2. Validación	3. Autenticación	4. Persistencia	5. Redirección
Email y contraseña	Zod devuelve errores específicos	Se compara con cuentas demo/registradas	Se guarda UserSession	El layout abre pestañas

Validaciones implementadas

Formulario	Reglas	Retroalimentación
Login	Correo válido; contraseña mínima de 6 caracteres	Mensaje bajo campo o error de credenciales
Registro	Nombre; correo; rol; 8 caracteres; confirmación; consentimiento	No permite duplicados y muestra la causa
Recuperación	Correo electrónico válido	Confirma que la solicitud fue procesada
Fotografía	Imagen obligatoria; tamaño máximo de 10 MB	Alerta contextual y retorno tras guardar

Decisiones técnicas

- React Hook Form mantiene el estado de formularios con pocas actualizaciones y API centrada en validación (React Hook Form, s. f.).
- Zod define esquemas TypeScript-first e infiere tipos estáticos desde las reglas (Zod, s. f.).
- El objeto de sesión excluye la contraseña antes de persistirse.
- Los layouts protegen rutas: sin usuario se redirige a /login; con sesión se habilitan las pestañas.

Límite de seguridad
Las cuentas registradas localmente son una simulación académica y sus credenciales no deben utilizarse en producción. La siguiente fase debe migrar a Firebase Authentication o un backend con hash, control de acceso y transporte seguro.

8. Navegación y módulos funcionales

La navegación principal usa cuatro pestañas persistentes: Inicio, Plan/Pacientes, Progreso y Perfil. La etiqueta de la segunda pestaña cambia según el rol. La carga de fotografía se presenta como ruta secundaria para concentrar la atención sin saturar la barra inferior.

Ruta	Paciente	Nutricionista	Lógica principal
/	Constancia, peso, cita y plan del día	Consultas, fotos y agenda	Render condicional por user.role
/plan	Comidas y acceso a fotografía	Listado de pacientes y planes	Contenido según rol
/progress	Peso, meta, IMC y tendencia	Estado resumido de pacientes	Datos demo y presentación comparativa
/profile	Datos y cierre de sesión	Datos profesionales y cierre	logout elimina la sesión
/upload-photo	Imagen, tipo y comentario	No corresponde al flujo diario	Permiso, selección y validación

Flujo de fotografía

Expo ImagePicker abre la interfaz del sistema para seleccionar imágenes o usar la cámara y funciona en Android, iOS y web (Expo, s. f.-e). NutriCare solicita permiso con un texto comprensible, permite editar la imagen, reduce calidad a 0.8 para el prototipo y rechaza archivos mayores de 10 MB. El comentario se limita a 240 caracteres.

Plan -> **Elegir imagen** -> **Revisar + comentar** -> **Confirmar**

Robustez demostrable

- La aplicación distingue cancelación, falta de permiso, ausencia de foto y exceso de tamaño.
- Los botones tienen estado deshabilitado/carga y retroalimentación visual al presionarse.
- La navegación no depende de variables globales sueltas; la sesión vive en un proveedor único.

9. Diseño multiplataforma, usabilidad y accesibilidad

Una interfaz móvil profesional no solo debe verse bien: debe responder a pantallas, entradas y capacidades diversas. React Native utiliza Flexbox y componentes respaldados por las plataformas; Expo Router añade navegación optimizada y universal (Meta Platforms, s. f.-b; Expo, s. f.-a).

Práctica	Aplicación en NutriCare	Beneficio
Safe area	SafeAreaView en el contenedor de pantalla	Evita notch y zonas reservadas
Scroll vertical	Screen encapsula ScrollView	Formularios utilizables en pantallas pequeñas
Teclado	KeyboardAvoidingView	El teclado no tapa los campos
Controles táctiles	Botón principal con altura mínima de 54	Supera la referencia de 44 x 44 para interacción táctil
Texto y error	Etiquetas visibles y mensajes cercanos al campo	No depende solo del color
Contraste	Texto oscuro y verde profundo sobre superficies claras	Jerarquía y lectura
Estados	pressed, disabled y loading	Evita dobles envíos y comunica respuesta

Referencia de accesibilidad

W3C explica que objetivos táctiles mayores reducen activaciones involuntarias y propone 44 por 44 píxeles para el criterio mejorado de tamaño de objetivo (W3C, 2023). NutriCare usa 54 puntos de alto en el botón principal, además de separación suficiente entre acciones.

Próxima verificación
Para la entrega final se recomienda probar VoiceOver/TalkBack, escalado de fuente, contraste cuantitativo, orientación y dispositivos Android/iOS físicos. La accesibilidad es una práctica continua, no una casilla única.

10. Calidad técnica y evidencia de verificación

La calidad del avance se evaluó con comprobaciones reproducibles. No basta con que una pantalla abra en un único equipo: se revisaron tipos, rutas y capacidad de empaquetado.

Comprobación	Resultado	Qué demuestra
TypeScript móvil	Aprobado sin errores	Props, esquemas, rutas y contexto son compatibles
Expo export web	Aprobado	Metro resolvió 934 módulos y generó 17 bundles de rutas
TypeScript web existente	Aprobado	La incorporación del workspace móvil no rompió la versión web
Revisión de estructura	Aprobado	Rutas separadas de componentes, estado, datos y validaciones
Recursos	Limpieza realizada	Se retiraron archivos de demostración que no utiliza NutriCare

Casos de prueba para la demostración

Caso	Acción	Resultado esperado
Correo inválido	Enviar 'abc'	Error debajo de correo; no autentica
Credenciales incorrectas	Usar clave distinta	Mensaje global de credenciales
Paciente válido	paciente@nutricare.com / nutricare2026	Abre Inicio con plan y métricas
Nutricionista válido	nutricionista@nutricare.com / nutricare2026	Abre resumen profesional
Registro inconsistente	Contraseñas diferentes	Mensaje 'Las contraseñas no coinciden'
Sesión persistente	Cerrar y abrir tras login	La sesión se restaura
Foto ausente	Guardar sin elegir	Alerta 'Falta la fotografía'

Comandos de reproducción

```
cd mobile
pnpm install
pnpm start
# En la terminal de Expo: a = Android, w = web
# Comprobación estática: pnpm exec tsc --noEmit
```

11. Matriz de cumplimiento del nivel Estratégico

Criterio Estratégico	Evidencia concreta	Cómo defenderlo
Pantallas profesionales, completas y coherentes con Figma	Sistema de tokens, 8 rutas, vistas por rol, tarjetas, estados y navegación inferior	Comparar mockup y app: identidad consistente, comportamiento nativo
Componentes reutilizables y buenas prácticas multiplataforma	Screen, BrandHeader, FormField, PrimaryButton, SurfaceCard; SafeArea, KeyboardAvoidingView y Flexbox	Mostrar que un cambio del botón afecta a todas las pantallas
Funcionalidad completa y robusta	Login, registro, recuperación, sesión persistente, protección de rutas, rol y fotos	Ejecutar un error y luego un flujo exitoso
Supera mínimos y demuestra avance sólido	Carga de imágenes, IMC, contenido por rol, datos demo y límites de archivo	Explicar por qué son funciones vinculadas al problema
Documentación técnica de alta calidad	Teoría oficial, aplicación al código, límites, pruebas, arquitectura y APA 7	Relacionar cada fuente con una decisión real
Justifica decisiones de diseño	Figma -> tokens -> componentes; matriz de correspondencia y accesibilidad	No decir 'porque se ve bonito': hablar de consistencia y uso

Autoevaluación honesta

Fortaleza actual
El avance es funcional, demostrable y coherente. La arquitectura permite ampliar sin rehacer las pantallas.

Límite actual
La autenticación local y los datos demo son adecuados para este hito, pero no sustituyen Firebase ni reglas de seguridad en la entrega integrada.

Evidencia que debe acompañar la entrega

- Repositorio GitHub con carpeta mobile, README y archivo de bloqueo.
- Video máximo de 15 minutos mostrando errores, flujo por ambos roles y selección de fotografía.
- Este PDF adjunto, con referencias APA 7.

12. Guion técnico de demostración

Guion sugerido para una explicación clara de 5 a 7 minutos dentro del video de entrega.

Tiempo	Qué mostrar	Qué explicar
0:00-0:40	Objetivo y login	NutriCare centraliza plan, progreso y seguimiento; móvil prioriza acceso cotidiano
0:40-1:30	Error de formulario	Zod valida el dato; React Hook Form conecta errores con cada campo
1:30-2:15	Login paciente	AuthContext busca cuenta, elimina contraseña de la sesión y persiste estado
2:15-3:20	Inicio, Plan y Progreso	Expo Router organiza rutas; componentes y tokens mantienen coherencia con Figma
3:20-4:10	Subir fotografía	ImagePicker pide permiso, valida tamaño y conserva una acción explícita
4:10-4:50	Rol nutricionista	La misma estructura adapta contenido mediante user.role
4:50-5:30	Código y arquitectura	UI, lógica y datos están separados; mostrar carpetas y componentes
5:30-6:00	Calidad y siguiente fase	TypeScript/export aprobados; Firebase será la persistencia segura compartida

Frases clave para la defensa

- 'React Native no copia HTML: renderiza componentes respaldados por la plataforma, pero conserva el modelo mental de React.'
- 'Figma fue la fuente del sistema visual; en código lo convertí en tokens y componentes reutilizables.'
- 'Expo Router convierte la estructura de archivos en navegación, por eso cada pantalla tiene una ruta clara.'
- 'La validación ocurre antes de la lógica de autenticación; así separo calidad del dato y credenciales.'
- 'AsyncStorage es deliberadamente temporal y no cifrado; la versión productiva migrará autenticación y datos a Firebase.'

Preparación antes de grabar

- Cerrar sesiones previas, iniciar Expo y tener visibles los dos accesos demo.
- Hacer la prueba completa una vez antes de grabar.
- Mostrar solamente archivos relevantes: app, components/ui, context, schemas y constants.

13. Riesgos, ética y hoja de ruta

NutriCare maneja información relacionada con salud. Aunque este avance usa datos de demostración, la evolución técnica debe aplicar minimización de datos, consentimiento, control de acceso y protección de archivos. El propósito es apoyar el seguimiento profesional, no diagnosticar ni reemplazar una consulta médica.

Prioridad	Mejora	Criterio de aceptación
Alta	Firebase Authentication	Contraseñas fuera del dispositivo y sesión segura
Alta	Cloud Firestore con reglas por rol	Paciente solo lee su expediente; nutricionista accede a asignados
Alta	Firebase Storage para fotografías	Archivos privados, URL controlada y metadatos mínimos
Media	Pruebas automatizadas	Cobertura de esquemas, contexto y rutas críticas
Media	Accesibilidad en dispositivo	TalkBack/VoiceOver, escalado y contraste documentados
Media	Sincronización web-móvil	Plan, mediciones y comentarios consistentes entre plataformas
Baja	Notificaciones	Recordatorios configurables sin exponer datos clínicos

Conclusiones

- React Native permitió convertir la planificación de la Fase 1 en una experiencia móvil real usando conceptos compartidos con React.
- Expo Router y TypeScript redujeron ambigüedad en la navegación y en los contratos del código.
- Figma aportó un lenguaje visual que se implementó como tokens y componentes, no como valores dispersos.
- Las validaciones y los estados de error convierten los formularios en funcionalidad verificable.
- El avance cumple los requisitos mínimos y agrega evidencia de robustez: roles, persistencia, fotografías, IMC, accesibilidad básica y compilación completa.

Conclusión central
El valor técnico de NutriCare no está en una pantalla aislada, sino en la relación coherente entre problema, diseño, arquitectura, interacción y evidencia de calidad.

14. Referencias

Formato APA 7. Fuentes oficiales consultadas el 18 de octubre de 2026.

Expo. (s. f.-a). Introduction to Expo Router. https://docs.expo.dev/router/introduction/

Expo. (s. f.-b). Core concepts of file-based routing in Expo Router. https://docs.expo.dev/router/basics/core-concepts/

Expo. (s. f.-c). Using TypeScript. https://docs.expo.dev/guides/typescript/

Expo. (s. f.-d). Store data. https://docs.expo.dev/develop/user-interface/store-data/

Expo. (s. f.-e). Expo ImagePicker. https://docs.expo.dev/versions/latest/sdk/imagepicker/

Figma. (s. f.-a). Guide to styles in Figma. https://help.figma.com/hc/en-us/articles/360039238753-Guide-to-styles-in-Figma

Figma. (s. f.-b). Lesson 3: Build your design system. https://help.figma.com/hc/en-us/articles/14548865734679-Lesson-3-Build-your-design-system

Meta Platforms. (s. f.-a). React fundamentals. React Native. https://reactnative.dev/docs/intro-react

Meta Platforms. (s. f.-b). Core components and APIs. React Native. https://reactnative.dev/docs/components-and-apis

Oviedo Martínez, F. Y. (2026). NutriCare: Proyecto Fase 1 [Proyecto académico no publicado]. Universidad Don Bosco.

React Hook Form. (s. f.). Performant, flexible and extensible forms. https://react-hook-form.com/

World Wide Web Consortium. (2023). Understanding success criterion 2.5.5: Target size (enhanced). https://www.w3.org/WAI/WCAG22/Understanding/target-size-enhanced

Zod. (s. f.). TypeScript-first schema validation with static type inference. https://zod.dev/

Nota metodológica
La documentación oficial se contrastó con el código entregado. Las afirmaciones sobre funcionalidades corresponden a archivos y verificaciones locales del proyecto NutriCare al 18 de octubre de 2026.